Scientific computing II 2026

Exercise 3
Submit your solution to Moodle no later than Thursday 19.11.2026 23:59

Note: Name even the source code files provided for you according to the exercise return format (for example, in this exercise `array_access.f90 → p5.f90`*).*

Problem 1.

The floating point number model of modern hardware also has a signed infinity. Write a program that does an arithmetic operation that produces a positive and a negative infinity and calculate the products $(+\infty) \times (-\infty)$ and $(-\infty) \times (-\infty)$.

Note: At least `gfortran` *does not like if you write a literal constant 1.0/0.0. Assigning the numerator and denominator values to variables and the doing the division works.*

Problem 2.

The floating point number model also has an object called 'not a number' (`NaN`). Arithmetic operations that are undefined produce `NaN`'s. For example, a square root of a negative number (when working with real numbers) produces a `NaN` as does division of zero by zero. In order to detect whether a real variable has a value `NaN` you can rely on the fact that `NaN` is never equal to `NaN` (yes, that sounds weird); i.e. you can use the if statements

```
if (x/=x) then
     print *,'We have a NaN!'
end if
```

to check whether `x` has a value `NaN`. Write a program that produces a `NaN` in two ways: taking a square root of a real variable and dividing zero by zero. Demonstrate the fact that `NaN` is never equal to itself.

Problem 3.

In Fortran *rank* of an array is the number of dimensions or the number of indices needed to refer to one element (lecture notes 'Arrays', slide 4). Maximum rank in Fortran90 is 7. Write Fortran statements that define a rank 7 integer array having size 2 in each direction. Intrinsic function `size(a)` returns the number of elements in array `a`, and `shape(a)` returns the size of `a` in each dimension. Demonstrate the use of these functions by printing their values when called with the above mentioned array as argument.

Problem 4.

Write a program to read the following data into a two-dimensional integer array:

```
1   2   3
4   5   6
```

```
7   8   9
```

Calculate totals for each row and column and produce output similar to that shown below:

```
1   2  3  6
4   5  6 15
7   8  9 24
12 15 18
```

Try to avoid excess use of `do` loops and use array slices and the `sum` function.

Problem 5.

Compile (using the optimization option `-O2`) and run the attached program `array_access.f90`. Comment on the output.

Program measures the CPU time used for execution using the intrinsic subroutine `cpu_time(t)`. It assigns the so far used time to the real argument `t`. So, measuring the time used in certain part of the code can be done as

```
call cpu_time(t1)
… ! Do something that consumes cpu time
call cpu_time(t2)
cpu=t2-t1
```

Hint1: You need not care about the value of `maxval(a)`*, it is suppposed to be different for the two cases. Pay attention to the printed value of variable* `cpu` *and the minor differences between the two loops.*
Hint2: Lecture notes Arrays, slide 18.

Problem 6. (1 point for (a) and (b) each)

Attached file `Kumpula_daily_temperature.dat` contains the time series of daily mean air temperature in Kumpula in time period of 12.11.2022–11.11.2024[1]. The first line in the file is the number of data points (number of lines in the file after the first line). After this every line contains just the temperature as a real number.

(a) Write a Fortran program that calculates the mean temperature of this time period. *It should first read the data into an array and then do the calculation.* Your program should assume the format mentioned above: 1st line contains the number of data points, and the rest are the data points. Note that the program does not know the number of data points before running it. This means that you must use an allocatable array.

(b) Calculate the weekly averages of the temperature. You may assume that the first week starts from the first data point in the file. To make your life easier, the number of data points is divisible by 7. Plot the original data and the weekly averages (they may be in separate plots so no need to get the x axis equal).

1 This kind of data can be freely downloaded from https://www.ilmatieteenlaitos.fi/havaintojen-lataus in excel or csv formats.

Hint1: Reading data from a file can be done using shell IO redirection.
Hint2: In `do` *loops you may have non-unit step (stride).*